TODO — Conception d'un SIRH (cycle de vie du salarié)

<!-- GLOSSAIRE - SIRH : Système d'Information des Ressources Humaines. Logiciel qui gère les salariés de l'embauche au départ. - RH : Ressources Humaines. - BDD : Base De Données. - PK / FK : Primary Key (identifiant unique d'une ligne) / Foreign Key (clé étrangère, lien vers la PK d'une autre table). - CRUD : Create, Read, Update, Delete. Les 4 opérations de base sur une donnée. Le cours dit : ne pas se limiter à de simples écrans CRUD, il faut des processus (workflows) et des règles métier. - MVP : Minimum Viable Product. Première version minimale mais utilisable. - CDD / CDI : Contrat à Durée Déterminée / Indéterminée. - Workflow : enchaînement d'étapes et de validations (ex. Manager -> RH -> Direction). - Onboarding : processus d'intégration d'un nouveau salarié (badge, compte, matériel, formation...). - Offboarding : processus de sortie (non montré dans les slides, déduit du cycle "Rupture"). - Matricule : numéro unique d'identification d'un salarié. - Affectation : rattachement d'un salarié à un poste et un département à une période donnée. - GED : Gestion Électronique des Documents (stockage des contrats, pièces justificatives). - BI : Business Intelligence (tableaux de bord, statistiques). - API REST : interface HTTP qui permet au front de parler au back. - Audit & logs : traces de qui a fait quoi et quand. - MVC / Front / Back : le Front est ce que l'utilisateur voit, le Back contient la logique et accède à la BDD. CYCLE GLOBAL (slide 1) Besoin -> Recrutement -> Embauche -> Vie du salarié -> Évolution -> Rupture Les blocs 1 à 4 couvrent Besoin, Recrutement, Embauche et Contrat/Onboarding. Les 3 dernières étapes (Vie, Évolution, Rupture) restent à concevoir. -->

Objectifs pédagogiques

- [ ] Comprendre les processus RH avant de concevoir les fonctionnalités
- [ ] Transformer un processus métier en règles de gestion, données, workflows et interfaces
- [ ] Concevoir un SIRH couvrant tout le cycle de vie du salarié
- [ ] Éviter la conception limitée à de simples écrans CRUD
- [ ] Faire évoluer un même cas pratique jusqu'à l'architecture et au MVP

Bloc 1 — Expression du besoin en personnel

<!-- Un manager exprime un besoin (création d'un nouveau poste ou remplacement). La demande suit une chaîne de validation : Manager -> RH -> Direction. Si tout est validé, le recrutement s'ouvre. POINT DE CONCEPTION : un POSTE existe indépendamment de l'EMPLOYÉ. Un poste peut être vacant, et un employé peut changer de poste. Donc on ne met jamais les infos du poste directement dans la table employé. -->

BDD

- [ ] Table `departement` (id PK, nom, responsable_id)
- [ ] Table `poste` (id PK, intitule, departement_id FK, description) — indépendante de `employe`
- [ ] Table `demande_recrutement` (id PK, poste_id FK, demandeur_id FK, effectif, motif [creation/remplacement], budget, date_souhaitee, statut)

- [ ] Table `validation` (id PK, demande_id FK, valideur_id FK, niveau [RH/Direction], decision, commentaire, date_decision)
- [ ] Contraintes : effectif > 0, budget >= 0, statut dans une liste définie

Métier (règles de gestion)

- [ ] Un manager ne peut créer une demande que pour son département
- [ ] Le motif est obligatoire (création ou remplacement)
- [ ] Ordre de validation imposé : Manager -> RH -> Direction
- [ ] Un refus à une étape bloque la suite et notifie le demandeur
- [ ] Le recrutement ne s'ouvre qu'une fois la demande validée par la Direction
- [ ] Statuts : brouillon, soumise, validée RH, validée Direction, refusée

Affichage

- [ ] Formulaire de demande de recrutement (poste, département, effectif, motif, budget, date)
- [ ] Liste des demandes avec filtre par statut
- [ ] Écran de validation pour RH et Direction (valider / refuser + commentaire)
- [ ] Suivi visuel de l'avancement de la demande

Bloc 2 — Recrutement

<!-- Une demande validée devient une offre d'emploi. L'offre est publiée, plusieurs candidatures arrivent. Les RH présélectionnent, organisent entretiens et tests, puis on conserve les évaluations et la décision finale. Relations clés : 1 offre -> N candidatures ; 1 candidat -> N candidatures ; 1 candidature -> N entretiens. Candidat et Candidature sont deux objets distincts (la même personne peut postuler plusieurs fois). -->

BDD

- [ ] Table `offre_emploi` (id PK, demande_id FK, titre, description, date_publication, date_cloture, statut)
- [ ] Table `candidat` (id PK, nom, prenom, email, telephone, cv_document_id)
- [ ] Table `candidature` (id PK, offre_id FK, candidat_id FK, date_depot, statut)
- [ ] Table `entretien` (id PK, candidature_id FK, type [entretien/test], date, intervenant_id FK)
- [ ] Table `evaluation_candidat` (id PK, entretien_id FK, note, commentaire, avis)
- [ ] Contrainte d'unicité : (offre_id, candidat_id) pour éviter les doublons de candidature

Métier

- [ ] Une offre ne peut être créée que depuis une demande validée
- [ ] Statuts de candidature : reçue, présélectionnée, entretien, test, retenue, refusée
- [ ] Impossible de planifier un entretien pour une candidature refusée

- [ ] Conserver toutes les évaluations (historique, pas d'écrasement)
- [ ] Décision finale obligatoire avant clôture de l'offre
- [ ] Respect des données personnelles des candidats (durée de conservation)

Affichage

- [ ] Page de création / publication d'offre
- [ ] Liste des candidatures par offre avec tri et présélection
- [ ] Planning des entretiens et tests
- [ ] Fiche d'évaluation du candidat
- [ ] Vue comparative des candidats pour la décision finale

Bloc 3 — Sélection et embauche

<!-- Le candidat retenu reçoit une proposition d'embauche. Après acceptation, on crée le dossier salarié et le matricule, puis le contrat, le poste et l'affectation. POINT DE CONCEPTION : CANDIDAT != EMPLOYÉ. Une personne peut avoir plusieurs candidatures avant de devenir salariée. À l'embauche, on crée un nouvel objet `employe` lié au `candidat` d'origine, on ne transforme pas la ligne candidat. -->

BDD

- [ ] Table `proposition_embauche` (id PK, candidature_id FK, salaire_propose, date_debut, date_envoi, statut)
- [ ] Table `employe` (id PK, matricule UNIQUE, candidat_id FK nullable, nom, prenom, date_embauche, statut)
- [ ] Génération automatique du matricule (unique, non réutilisable)
- [ ] Création liée : `employe` + `contrat` + `affectation` dans une même transaction

Métier

- [ ] Proposition possible uniquement pour une candidature "retenue"
- [ ] Si la proposition est refusée, la candidature repasse en attente ou refusée
- [ ] À l'acceptation : créer dossier salarié, matricule, contrat, affectation (tout ou rien)
- [ ] Le candidat garde son historique de candidatures après l'embauche

Affichage

- [ ] Écran d'envoi / suivi de la proposition d'embauche
- [ ] Bouton "Convertir en salarié" après acceptation
- [ ] Fiche salarié créée avec matricule affiché
- [ ] Récapitulatif contrat, poste, affectation

Bloc 4 — Contrat et onboarding

<!-- Un salarié peut avoir plusieurs contrats successifs (CDD, renouvellement, puis CDI...). Il ne faut pas écraser l'historique en stockant seulement `employe.type_contrat` : si on modifie ce champ, on perd les contrats précédents. Il faut donc une table `contrat` séparée avec dates de début et de fin. L'onboarding est géré avec une checklist et un workflow d'intégration (contrat signé, badge, compte, matériel, formation, inscription paie). -->

BDD

- [ ] Table `contrat` (id PK, employe_id FK, type [CDD/CDI/...], date_debut, date_fin, salaire, statut)
- [ ] Supprimer `type_contrat` de la table `employe` (ou le dériver du contrat actif)
- [ ] Table `affectation` (id PK, employe_id FK, poste_id FK, departement_id FK, date_debut, date_fin)
- [ ] Table `onboarding` (id PK, employe_id FK, date_debut, statut)
- [ ] Table `onboarding_tache` (id PK, onboarding_id FK, libelle, responsable_id FK, echeance, statut)
- [ ] Table `document` (id PK, employe_id FK, type, chemin_stockage, date_depot)

Métier

- [ ] Un salarié peut avoir plusieurs contrats, mais un seul contrat actif à la fois
- [ ] Pas de chevauchement de dates entre deux contrats d'un même salarié
- [ ] Un renouvellement crée un nouveau contrat, il ne modifie pas l'ancien
- [ ] CDD : date de fin obligatoire ; CDI : pas de date de fin
- [ ] Modèle de checklist onboarding : contrat signé, badge, compte, matériel, formation, inscription paie
- [ ] Onboarding terminé uniquement quand toutes les tâches obligatoires sont faites

Affichage

- [ ] Historique des contrats sur la fiche salarié (frise chronologique)
- [ ] Formulaire de création / renouvellement de contrat
- [ ] Checklist d'onboarding avec cases à cocher et responsable par tâche
- [ ] Dépôt et consultation des documents (contrat signé, pièces)
- [ ] Indicateur d'avancement de l'intégration (ex. 4/6 tâches)

Étapes restantes du cycle (à concevoir)

<!-- Annoncées dans le schéma du cycle mais pas détaillées dans les slides. À compléter selon la suite du cours. -->

Vie du salarié

- [ ] BDD : congés, absences, formations, paie (à définir)
- [ ] Métier : règles de congés, validation des absences

- [ ] Affichage : espace salarié

Évolution

- [ ] BDD : promotions, changements d'affectation (historique via `affectation`)
- [ ] Métier : règles de mobilité interne
- [ ] Affichage : historique de carrière

Rupture

- [ ] BDD : table de sortie (motif, date, type : démission, fin de CDD, licenciement...)
- [ ] Métier : clôture du contrat actif, offboarding (restitution matériel, fermeture comptes)
- [ ] Affichage : processus de départ avec checklist

Architecture technique possible

<!-- Couche applicative : Front Web/Mobile (interface) -> API REST (communication) -> Services métier RH (règles de gestion) -> Moteur de workflow (validations, onboarding) -> Authentification (qui peut faire quoi). Données & intégration : PostgreSQL/MySQL (BDD relationnelle), GED (documents), Notifications (mails, alertes), Audit & logs (traçabilité), Reporting/BI (statistiques). -->

- [ ] Choisir la BDD : PostgreSQL ou MySQL
- [ ] Mettre en place l'authentification et les rôles (manager, RH, direction, salarié)
- [ ] Exposer une API REST par domaine (besoin, recrutement, contrat, onboarding)
- [ ] Implémenter les services métier RH (règles de gestion)
- [ ] Ajouter un moteur de workflow (validations et checklists)
- [ ] Prévoir GED / stockage des documents
- [ ] Prévoir les notifications (validation en attente, tâche d'onboarding)
- [ ] Journaliser les actions (audit & logs)
- [ ] Prévoir reporting / BI

MVP

- [ ] Définir le périmètre minimal : Bloc 1 à 4 de bout en bout
- [ ] Un parcours complet : demande -> offre -> candidature -> embauche -> contrat -> onboarding